CODING · AULA 11 · 27 DE OUTUBRO DE 2026

A loja ganha vitrine

Entrega 1 do projeto e o começo do front-end: HTML semântico, acessibilidade e CSS com Flexbox e Grid.

Faculdade Senac Pernambuco · ADS 2026.2 · Prof. Sebastião Rogério

ONDE ESTAMOS NO SEMESTRE · 1 DE 3

O caminho até aqui

U1 · ✓ CONCLUÍDA

Fundamentos

11/08
Por que tantas linguagens

18/08
Paradigmas

25/08
Tipos e coleções

01/09
Funções, módulos e testes

U2 · ✓ CONCLUÍDA

Orientação a objetos

08/09
Classes

22/09
AV1 e encapsulamento

29/09
Herança e polimorfismo

06/10
Contratos e a Loja de Roupas

U3 · ✓ CONCLUÍDA

Back-end e API

13/10
HTTP, REST e a primeira API

20/10
Persistência e camadas

U4 · ◂ COMEÇA HOJE

Front-end

27/10
Hoje: ENTREGA 1, HTML e CSS

03/11
JavaScript e eventos

10/11
Integração com a API

U5

Entrega e avaliação

17/11
PROJETO FINAL

24/11
AV2

01/12
Framework (sem nota)

08/12
Encerramento

ONDE ESTAMOS NO SEMESTRE · 2 DE 3

A aplicação que estamos construindo, camada por camada

Telas no navegador	HTML e CSS hoje; JavaScript depois · Aulas 11 a 13	◂ HOJE
▲ fetch (Aula 13) ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✔ pronto
▲ chama ▼		
Serviço e repositório	as camadas entre a porta e o banco · Aula 10	✔ pronto
▲ usa ▼		
Domínio e banco	as classes e o loja.db · Aulas 5 a 10	✔ pronto

- O **back-end** da loja está pronto: API, regras e banco.
- Hoje começa a camada que o cliente **vê**: as telas.
- Nesta aula as telas ainda são **estáticas**. Na Aula 13 elas passam a buscar os dados na API.

ONDE ESTAMOS NO SEMESTRE · 3 DE 3

O que falta: as notas fecham em 27/11

DATA	AULA	TEMA	O QUE ACRESCENTA AO PROJETO
27/10	**11**	**ENTREGA 1 · HTML e CSS**	**hoje: a API é avaliada e nasce a primeira tela**
03/11	12	JavaScript, DOM e eventos	as telas reagem ao usuário
10/11	13	Assincronismo e integração	as telas conversam com a API
17/11	14	**PROJETO FINAL**	apresentação com demonstração: 50% do segundo processo
24/11	15	**AV2** e devolutiva	prova: 50% do segundo processo
01/12 e 08/12	16 e 17	Framework e encerramento	conteúdo complementar, sem nota

O QUE VAMOS VER HOJE

A API avaliada, e a primeira tela da loja

ABERTURA

Entrega 1

`apresentação das equipes`

- Demonstração rápida da API
- Checklist da Aula 10

MUNDO 1

HTML

`teoria com exemplos`

- A página por dentro
- HTML semântico
- Acessibilidade mínima

MUNDO 2

CSS

`teoria com exemplos`

- Seletores e cascata
- Modelo de caixa
- Flexbox, Grid e telas pequenas

MUNDO 3

Prática na loja

`vocês seguem a aula`

- A vitrine
- O visual
- O formulário de cadastro

ENTREGA 1 · COMO VAI SER

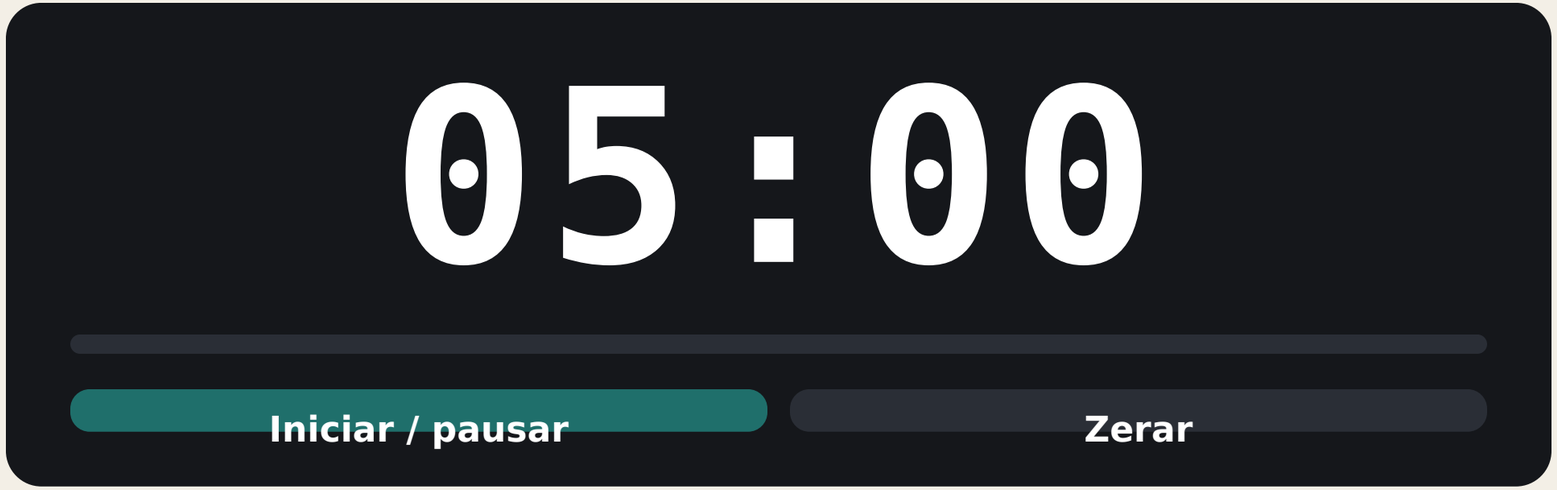
Cada equipe mostra a API, rapidinho

CRITÉRIO	O QUE A EQUIPE MOSTRA	PONTOS
API funcionando	a página `/docs` com o CRUD do recurso principal	2
Dados guardados	cadastrar, reiniciar o servidor, e o dado continua lá	2
Camadas	rota, serviço e repositório em arquivos separados	2
Testes	`pytest` verde e o ✓ do GitHub Actions	2
README	o que é o sistema e como rodar	1
Domínio do código	respostas às perguntas do professor	1

Uns 5 minutos por equipe: 3 de demonstração e 2 de perguntas. Enquanto uma apresenta, a próxima já deixa o servidor ligado.

ENTREGA 1

Vez da equipe



- Abrir a `/docs` e cadastrar um item.
- Desligar e ligar o servidor: o item continua lá.
- Mostrar as camadas no editor.
- Rodar o `pytest` e mostrar o Actions verde.



O MAPA DE HOJE

Nove fases, três mundos

Mundos 1 e 2: HTML e CSS, os conceitos, com exemplos. Mundo 3: a vitrine e o cadastro da Loja de Roupas, com checkpoints.



FASE 1-1 · CONTEÚDO NOVO

A página por dentro

O que o navegador recebe, e como transforma texto em tela.

CONCEITO AULA 11

As três linguagens do navegador

Toda página usa até três linguagens, cada uma com um papel. **HTML** diz **o que** há na página (estrutura e significado). **CSS** diz **como** aparece. **JavaScript** diz **o que acontece** quando o usuário age.

exemplo

```
HTML        o quê     título, lista, botão, formulário
CSS         como      cor, tamanho, posição, colunas
JavaScript  quando    clicou, digitou, chegou dado

hoje: HTML e CSS
Aulas 12 e 13: JavaScript
```

PENSE ASSIM

Uma casa: o HTML é a planta e as paredes, o CSS é a pintura e os móveis, o JavaScript é a instalação elétrica.

CONCEITO AULA 11

Elemento HTML

Um pedaço da página marcado por **tags**: a de abertura, o conteúdo e a de fechamento. **Atributos** dão informações extras dentro da tag de abertura. Elementos ficam **dentro** de outros, formando uma árvore.

exemplo

```
<a href="cadastro.html">Cadastrar produto</a>
 |  └ atributo      └ conteúdo        └ fecha
 └ tag de abertura

<article>
  <h3>Camiseta básica</h3>   <- dentro do article
  <p>R$ 39,90</p>
</article>
```

CUIDADO

Tag aberta tem que ser fechada, na ordem certa. `<p><strong>oi</p></strong>` está errado.

CONCEITO AULA 11

O documento HTML

Todo arquivo HTML tem o mesmo esqueleto. O `head` traz informações **sobre** a página (título da aba, codificação, CSS). O `body` traz o que **aparece**. O `lang` diz o idioma, para leitores de tela e tradutores.

exemplo

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Vitrine · Loja de Roupas</title>
</head>
<body>
  ... o que aparece ...
</body>
</html>
```

CUIDADO

Sem a meta `viewport`, o celular mostra a página "de longe", minúscula. Ela entra em toda página.

FASE 1-1 · PASSO A PASSO

Do arquivo à tela

1. **O navegador baixa**
   o arquivo index.html
2. **Monta a árvore**
   cada elemento vira um nó: o DOM
3. **Lê o CSS**
   e decide o estilo de cada nó
4. **Calcula o layout**
   tamanho e posição de cada caixa
5. **Desenha**
   os pixels na tela



FASE 1-2 · CONTEÚDO NOVO

HTML semântico

Dá para fazer tudo com div. Mas aí só os olhos entendem a página.

CONCEITO AULA 11

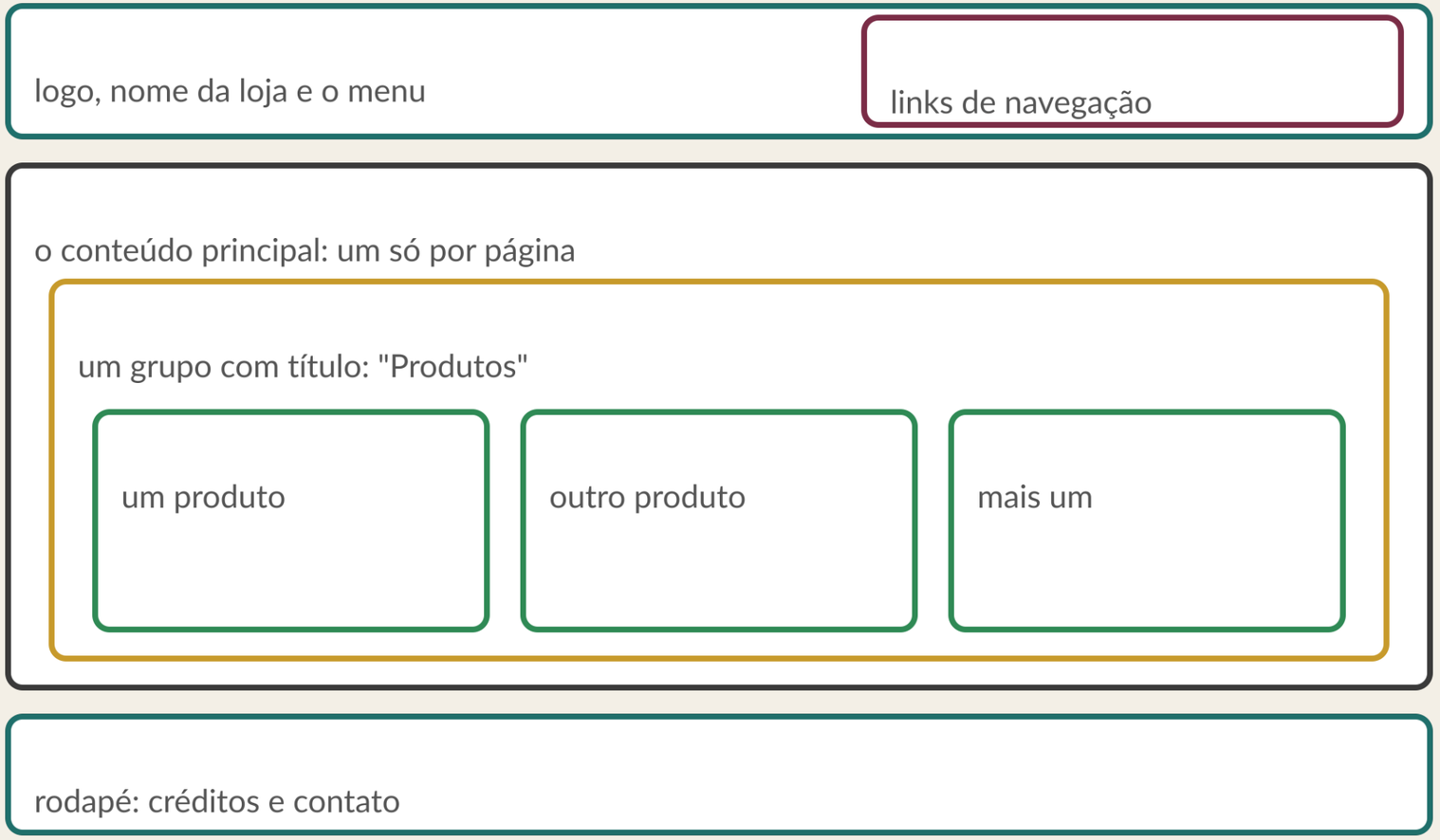
HTML semântico

Usar a tag que diz **o que** o conteúdo é, e não só **como** ele fica. `header`, `nav`, `main`, `section`, `article` e `footer` têm significado: o leitor de tela, o buscador e o próximo programador entendem a página sem ver.

exemplo

FASE 1-2 · NA IMAGEM

As regiões da vitrine da loja



FASE 1-2 · TABELA

Qual tag usar

TAG	USE PARA	NA LOJA
`header`	topo da página ou de uma seção	logo, nome e menu
`nav`	um grupo de links de navegação	Vitrine · Cadastrar produto
`main`	o conteúdo principal: **um só** por página	a vitrine
`section`	um grupo de conteúdo com título	"Produtos"
`article`	algo que faz sentido sozinho	cada produto
`h1` a `h6`	títulos, em ordem, sem pular nível	h1 Vitrine, h2 Produtos, h3 nome
`footer`	rodapé	créditos
`div`	agrupar só para o CSS, quando nenhuma acima serve	(quase nunca)

FASE 1-2 · QUAL TAG?

Escolha a tag mais adequada para cada caso.

- a) O menu com "Vitrine" e "Cadastrar produto".
- b) O cartão de um produto, com nome, preço e botão.
- c) O título "Vitrine", o mais importante da página.
- d) Um bloco só para pôr cor de fundo, sem significado.

a) nav, dentro do header.

b) article: faz sentido sozinho.

c) h1: um por página.

d) div: é o caso em que ela serve.



FASE 1-3 · CONTEÚDO NOVO

Acessibilidade

Uma página que só funciona para quem enxerga bem e usa mouse deixa clientes de fora.

CONCEITO AULA 11

Acessibilidade na web

Fazer a página funcionar para **todas as pessoas**, inclusive quem usa **leitor de tela**, só o **teclado** ou tem baixa visão. As regras estão na **WCAG**, a norma internacional do W3C. No Brasil, a Lei Brasileira de Inclusão (Lei 13.146/2015) exige sites acessíveis.

exemplo

```
o mínimo de toda página:

✔ alt em toda imagem
✔ label ligado a todo campo
✔ títulos em ordem (h1, h2, h3)
✔ contraste suficiente entre texto e fundo
✔ dá para usar só com o teclado (Tab)
```

PENSE ASSIM

A rampa ao lado da escada: foi pensada para cadeirantes, e todo mundo com carrinho ou mala acaba usando.

CONCEITO AULA 11

alt e label

O `alt` é o **texto da imagem**: o leitor de tela lê em voz alta. O `label` dá **nome ao campo**: o `for` do label aponta para o `id` do campo. Clicar no rótulo também passa a selecionar o campo.

exemplo

```
<img src="img/logo.svg"
     alt="Logotipo da Loja de Roupas: uma camiseta">

<label for="preco">Preço (R$)</label>
<input id="preco" name="preco" type="number">

<!-- imagem só decorativa: alt vazio -->
<img src="enfeite.svg" alt="">
```

CUIDADO

Placeholder não substitui label: ele some quando a pessoa começa a digitar, e muitos leitores de tela não o anunciam.

FASE 1-3 · DIAGNÓSTICO

O que um leitor de tela não consegue entender aqui?

trecho.html

```
<img src="camiseta.jpg">
<input type="text" placeholder="Nome">
<h1>Vitrine</h1>
<h4>Produtos</h4>
```

A imagem não tem `alt`: o leitor diz só "imagem" ou o nome do arquivo.

O campo não tem `label`: o placeholder some ao digitar.

Os títulos pulam do h1 para o h4: a estrutura fica confusa.



FASE 2-1 · CONTEÚDO NOVO

Seletores e cascata

Como o CSS escolhe em quais elementos aplicar cada estilo, e quem vence quando dois discordam.

CONCEITO AULA 11

Regra CSS

Uma regra tem um **seletor** (em quem aplicar) e uma lista de **declarações** `propriedade: valor`. O arquivo `.css` entra na página por um `link` no `head`.

exemplo

```
/* elemento: todos os botões */
button { background: #1f6f6b; color: #fff; }

/* classe: quem tem class="formulario" */
.formulario { max-width: 420px; }

/* descendente: links dentro do nav */
nav a { color: #fff; }

<link rel="stylesheet" href="estilo.css">
```

CUIDADO

Ponto antes do nome é **classe** (`.formulario`). Sem ponto é **elemento** (`button`). Trocar um pelo outro é o erro mais comum.

CONCEITO AULA 11

Cascata e especificidade

Quando duas regras mandam no mesmo elemento, vence a **mais específica**: id vence classe, que vence elemento. Se empatarem, vence a que vem **por último** no arquivo.

exemplo

```
p           { color: black; }   /* elemento: fraco */
.preco      { color: green; }   /* classe: médio   */
#destaque   { color: red; }     /* id: forte       */

<p class="preco" id="destaque">R$ 39,90</p>
<!-- fica vermelho -->
```

PENSE ASSIM

Uma ordem do gerente vale mais que um aviso no mural; entre dois avisos do mesmo nível, vale o mais recente.

FASE 2-1 · QUEM VENCE?

De que cor fica o botão?

estilo.css

```
button { background: gray; }
.comprar { background: green; }
button { background: blue; }

<button class="comprar">Pôr no carrinho</button>
```

Verde. A classe `.comprar` é mais específica que o elemento `button`, então vence mesmo vindo antes.

Entre as duas regras de `button`, a azul viria por último, mas nenhuma das duas vence a classe.



FASE 2-2 · CONTEÚDO NOVO

Modelo de caixa

Para o CSS, todo elemento é uma caixa. Entender a caixa resolve metade dos problemas de layout.

CONCEITO AULA 11

Modelo de caixa

Todo elemento é uma caixa em camadas: o **conteúdo**, o **padding** (espaço por dentro), a **border** (a borda) e a **margin** (espaço por fora, até a próxima caixa).

exemplo

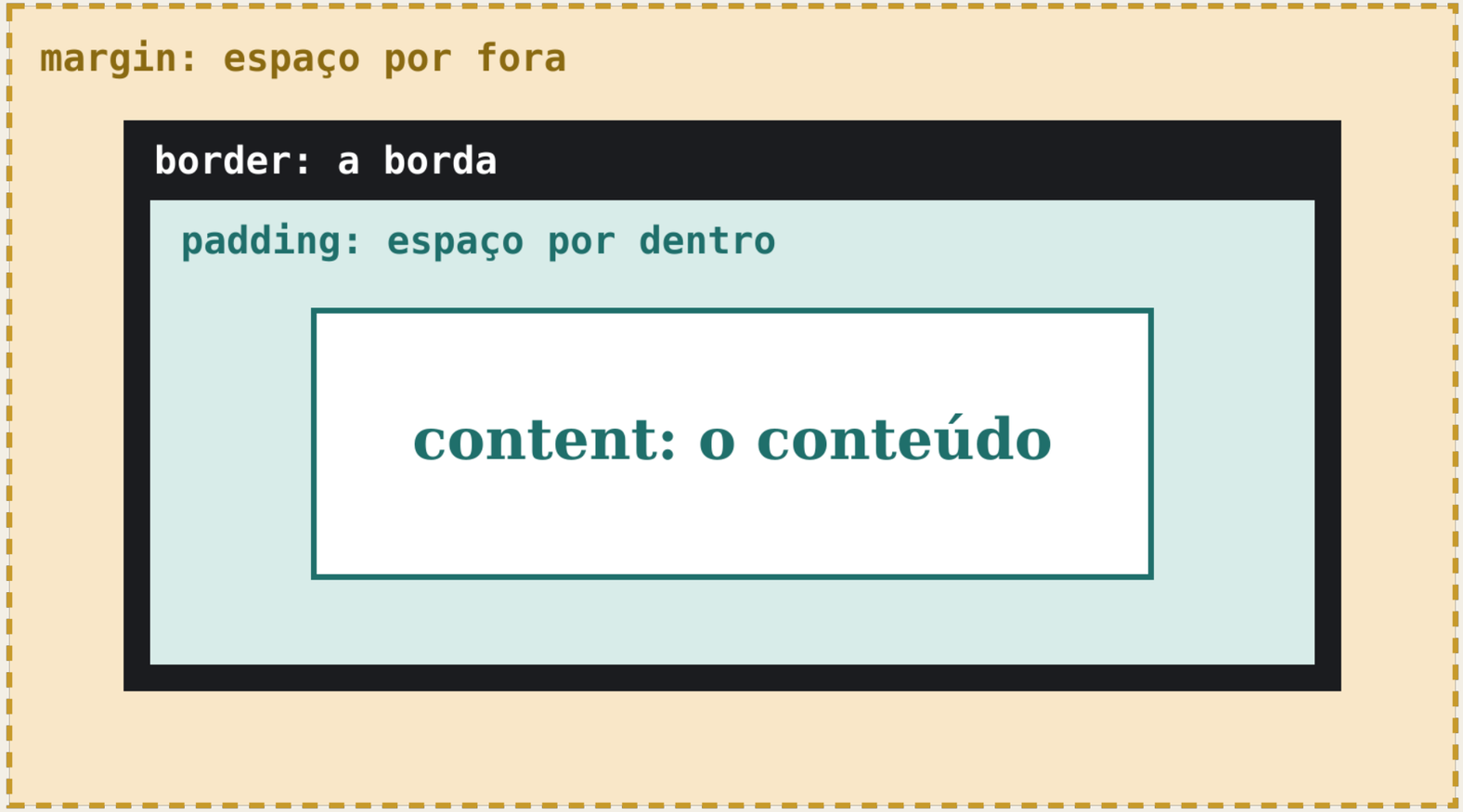
```
article {
  padding: 16px;              /* dentro  */
  border: 1px solid #e2dccd; /* borda   */
  border-radius: 12px;        /* cantos  */
  margin: 0 0 20px;           /* fora    */
}
```

CUIDADO

Por padrão, `width` mede só o conteúdo: padding e borda somam por fora. Com `box-sizing: border-box`, a largura inclui tudo. Use em todo projeto.

FASE 2-2 · NA IMAGEM

As quatro camadas de uma caixa



CONCEITO AULA 11

Variáveis CSS

Valores com nome, definidos uma vez em `:root` e usados com `var(...)`. Mudar a cor da marca vira mudar **uma linha**.

exemplo

```
:root {
  --verde: #1f6f6b;
  --borda: #e2dccd;
}

header { background: var(--verde); }
button { background: var(--verde); }
article { border: 1px solid var(--borda); }
```

PENSE ASSIM

As constantes do Python (`FRETE_FIXO = 15.0`), agora no CSS.



FASE 2-3 · CONTEÚDO NOVO

Flexbox e Grid

Duas ferramentas de layout: uma para enfileirar, outra para montar grades.

CONCEITO AULA 11

Flexbox

Organiza os filhos de um elemento **numa direção**: em linha (padrão) ou em coluna. `gap` dá o espaço entre eles, `align-items` alinha no outro eixo e `margin-left: auto` empurra um item para o fim.

exemplo

```
header {
  display: flex;          /* filhos lado a lado */
  align-items: center;    /* centralizados na altura */
  gap: 16px;
}
nav { margin-left: auto; }   /* menu vai para a direita */

.formulario {
  display: flex;
  flex-direction: column; /* um embaixo do outro */
}
```

PENSE ASSIM

Uma fila: todo mundo numa linha só, e você decide o espaço entre as pessoas e quem fica no fim.

CONCEITO AULA 11

Grid

Organiza os filhos em **linhas e colunas** ao mesmo tempo. `grid-template-columns` diz quantas colunas e de que largura; `1fr` é "uma fração do espaço que sobra".

exemplo

```
section {
  display: grid;
  grid-template-columns: repeat(3, 1fr);  /* 3 colunas iguais */
  gap: 20px;
}

/* o título ocupa a linha inteira */
section h2 { grid-column: 1 / -1; }
```

PENSE ASSIM

Uma prateleira com nichos do mesmo tamanho: cada produto cai no próximo nicho livre.

FASE 2-3 · COMPARAÇÃO

Flexbox ou Grid?

	FLEXBOX	GRID
Direções	uma: linha **ou** coluna	duas: linhas **e** colunas
Quem manda	o tamanho do conteúdo	a grade definida no pai
Bom para	menu, barra do topo, botões alinhados, formulário em coluna	vitrine de cartões, painel, a página inteira
Na loja	`header` e `.formulario`	a `section` de produtos

Não é um ou outro: a mesma página usa os dois, cada um onde faz sentido.

CONCEITO AULA 11

Telas pequenas (media query)

Um bloco de CSS que só vale **sob uma condição**, como a largura da tela. Assim a mesma página fica em 3 colunas no computador e em 1 no celular.

exemplo

```
@media (max-width: 700px) {
  section {
    grid-template-columns: 1fr;  /* uma coluna */
  }
  header { flex-wrap: wrap; }
}
```

CUIDADO

Teste no DevTools: o ícone de celular (Ctrl+Shift+M) simula telas de vários tamanhos.

HORA DA PRÁTICA

Agora é com vocês

1

Abrir esta aula

no site, em Materiais › Coding, na Fase 3-1 (tecla M abre o mapa)

2

Partir da etapa-15

o projeto da loja da Aula 10, ou a tag pronta

3

Criar a pasta front/

as telas ficam separadas do back-end

4

Abrir no navegador

duplo clique no index.html, ou a extensão Live Server do VS Code

5

Rodar o pytest

o test_front.py confere estrutura e acessibilidade

EM DUPLA

Um digita, o outro lê o slide e confere no navegador. Troquem a cada fase.



FASE 3-1 · PRÁTICA

A vitrine

A página principal da loja, só com HTML semântico: o mesmo catálogo da API.

FASE 3-1 · NO EDITOR

front/index.html: o esqueleto e o topo

front/index.html

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Vitrine · Loja de Roupas</title>
</head>
<body>
  <header>
    <img src="img/logo.svg" alt="Logotipo da Loja de Roupas: uma camiseta" width="48" height="48">
    <p>Loja de Roupas</p>
    <nav aria-label="Principal">
      <a href="index.html">Vitrine</a>
      <a href="cadastro.html">Cadastrar produto</a>
    </nav>
  </header>
```

- `lang`, `charset` e `viewport`: as três linhas de toda página.
- O logotipo tem `alt` que diz o que ele mostra.
- `aria-label` dá nome ao menu para o leitor de tela.

FASE 3-1 · NO EDITOR

front/index.html: a vitrine

front/index.html (continuação)

```
  <main>
    <h1>Vitrine</h1>
    <p>Promoção da semana: 10% em tudo. Frete grátis a partir de R$ 200.</p>

    <section aria-labelledby="titulo-produtos">
      <h2 id="titulo-produtos">Produtos</h2>

      <article>
        <h3>Camiseta básica</h3>
        <p>Tamanho M · manga curta</p>
        <p><strong>R$ 39,90</strong></p>
        <button type="button">Pôr no carrinho</button>
      </article>
      ...
```

- Um `h1`, depois `h2` e `h3`: títulos em ordem.
- `aria-labelledby` liga a seção ao título dela.
- Os três produtos são os mesmos da API. Na Aula 13, eles virão do `GET /produtos`.

FASE 3-1 · NO NAVEGADOR

Sem CSS: feio, mas já organizado

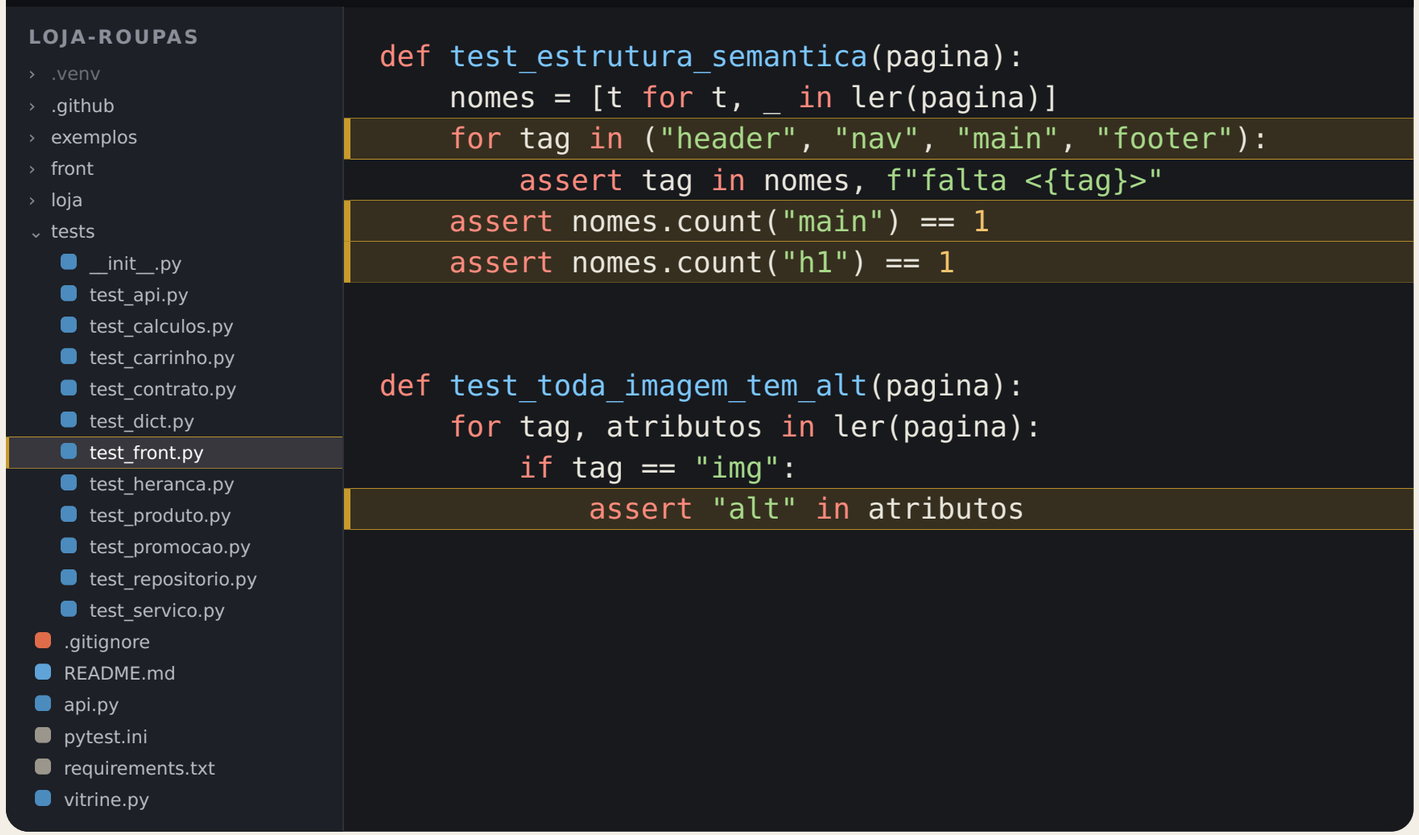
`front/index.html, sem estilo`
(a página aparece aqui na versão HTML da aula)

- O navegador já mostra títulos maiores, listas e botões: é o estilo **padrão**.
- A ordem e o significado estão certos. Agora é só vestir.

FASE 3-1 · NO EDITOR

tests/test_front.py: o pytest também confere a página

tests/test_front.py




```python
def test_estrutura_semantica(pagina):
    nomes = [t for t, _ in ler(pagina)]
    for tag in ("header", "nav", "main", "footer"):
        assert tag in nomes, f"falta <{tag}>"
    assert nomes.count("main") == 1
    assert nomes.count("h1") == 1


def test_toda_imagem_tem_alt(pagina):
    for tag, atributos in ler(pagina):
        if tag == "img":
            assert "alt" in atributos
```

- O `HTMLParser` do próprio Python lê a página e lista as tags.
- Cada página da pasta `front/` passa pelos mesmos testes (`parametrize`).
- Esqueceu o `alt`? O pytest e o GitHub Actions ficam vermelhos.

FASE 3-1 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
61 passed
$ git add .
$ git commit -m "Fase 3-1: vitrine em HTML semântico"
$ git push
```

CONFIRA

O `pytest` mostra **61 passed** e a vitrine abre no navegador, sem estilo.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-16.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-16`



FASE 3-2 · PRÁTICA

O visual

O mesmo HTML, agora com estilo: topo com Flexbox e vitrine com Grid.

FASE 3-2 · NO EDITOR

front/estilo.css: cores, caixa e o topo

front/estilo.css

```
/* Etapa 17: o visual da loja (Aula 11) */

:root {
  --verde: #1f6f6b;
  --tinta: #1b1c1f;
  --fundo: #f3efe6;
  --cartao: #fffdf8;
  --borda: #e2dccd;
}

* { box-sizing: border-box; }       /* largura inclui padding e borda */

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: var(--tinta);
  background: var(--fundo);
  line-height: 1.5;
}

/* cabeçalho: logo, nome e menu numa linha só (Flexbox) */
header {
  display: flex;
  align-items: center;
  gap: 16px;
  padding: 12px 24px;
  background: var(--verde);
  color: #fff;
}
header p { font-size: 1.3rem; font-weight: 700; margin: 0; }
nav { margin-left: auto; display: flex; gap: 20px; }
nav a { color: #fff; }
```

- As cores ficam em variáveis, no `:root`.
- `box-sizing` vale para tudo (`*`).
- O `header` vira uma fila (Flexbox), e o menu vai para a direita.
- No `index.html`, acrescente no `head`: `<link rel="stylesheet" href="estilo.css">`.

FASE 3-2 · NO EDITOR

front/estilo.css: a vitrine em grade

front/estilo.css (continuação)

```
main { max-width: 1000px; margin: 0 auto; padding: 24px; }

/* a vitrine: cartões em grade (Grid) */
section {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
section h2 { grid-column: 1 / -1; margin: 0; }

article {
  background: var(--cartao);
  border: 1px solid var(--borda);
  border-radius: 12px;
  padding: 16px;
}
article h3 { margin-top: 0; }

button {
  background: var(--verde);
  color: #fff;
  border: 0;
  border-radius: 8px;
  padding: 10px 16px;
  font-size: 1rem;
  cursor: pointer;
}
```

- `main` fica centralizado com largura máxima: `margin: 0 auto`.
- A `section` vira uma grade de 3 colunas iguais.
- Cada `article` é um cartão: fundo, borda, cantos e padding.

FASE 3-2 · NO NAVEGADOR

O mesmo HTML, vestido

`front/index.html com estilo.css`
(a página aparece aqui na versão HTML da aula)

FASE 3-2 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
61 passed
$ git add .
$ git commit -m "Fase 3-2: CSS com Flexbox e Grid"
$ git push
```

CONFIRA

A vitrine aparece em **3 colunas**, com o topo verde e o menu à direita. O `pytest` continua com **61 passed**.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-17.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-17`



FASE 3-3 · PRÁTICA

Formulário

A tela de cadastro, com os campos do POST /produtos, acessível e boa no celular.

FASE 3-3 · NO EDITOR

front/cadastro.html: o formulário

front/cadastro.html

```
<form class="formulario">
  <label for="tipo">Tipo</label>
  <select id="tipo" name="tipo">
    <option value="Produto">Produto</option>
    <option value="Camiseta">Camiseta</option>
    <option value="Calca">Calça</option>
  </select>

  <label for="nome">Nome</label>
  <input id="nome" name="nome" type="text" required>

  <label for="preco">Preço (R$)</label>
  <input id="preco" name="preco" type="number" min="0.01" step="0.01" required>

  <label for="tamanho">Tamanho</label>
  <select id="tamanho" name="tamanho" required>
    <option>PP</option><option>P</option><option selected>M</option>
    <option>G</option><option>GG</option>
  </select>

  <button type="submit">Cadastrar</button>
</form>
```

- Cada `label for` aponta para o `id` do campo.
- Os `name` são os mesmos campos do `POST /produtos`.
- `required` e `min` já validam no navegador. O servidor continua validando: o navegador ajuda, não garante.
- O topo e o rodapé são os mesmos do `index.html`.

FASE 3-3 · NO EDITOR

front/estilo.css: formulário e celular

front/estilo.css (final do arquivo)

```
/* Etapa 18: formulário e telas pequenas */

.formulario {
  display: flex;
  flex-direction: column;     /* um campo embaixo do outro */
  gap: 8px;
  max-width: 420px;
}
.formulario input,
.formulario select {
  font-size: 1rem;
  padding: 10px;
  border: 1px solid var(--borda);
  border-radius: 8px;
}
.formulario button { margin-top: 12px; align-self: flex-start; }

/* celular: uma coluna de cartões e menu embaixo do nome */
@media (max-width: 700px) {
  section { grid-template-columns: 1fr; }
  header { flex-wrap: wrap; }
  nav { margin-left: 0; width: 100%; }
}
```

- O formulário é uma coluna (Flexbox).
- Abaixo de 700 px, a vitrine vira **uma** coluna e o menu desce.
- Teste no DevTools com o modo celular (Ctrl+Shift+M).

FASE 3-3 · NO NAVEGADOR

A tela de cadastro

`front/cadastro.html`
(a página aparece aqui na versão HTML da aula)

FASE 3-3 · PASSO A PASSO

Conferir a página como um profissional

1. **Validador do W3C**
   validator.w3.org › Validate by File Upload: zero erros
2. **DevTools › Lighthouse**
   marcar só "Acessibilidade" e rodar a análise
3. **Meta: 90 ou mais**
   cada item reprovado diz o elemento e como corrigir
4. **Teclado**
   usar só o Tab e o Enter: dá para cadastrar sem mouse?
5. **pytest**
   o test_front.py confere os labels e os campos

FASE 3-3 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
66 passed
$ git add .
$ git commit -m "Fase 3-3: formulário acessível e responsivo"
$ git push
```

CONFIRA

O `pytest` mostra **66 passed**, o validador não acusa erros e o Lighthouse dá **90 ou mais** em acessibilidade.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-18.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-18`

LABORATÓRIO DA EQUIPE

A primeira tela do projeto de vocês

O QUE FAZER

Duas páginas em front/

1. A **listagem** do recurso principal, com cartões em Grid. 2. O **formulário de cadastro**, com os mesmos campos do `POST` da API de vocês. Topo e rodapé iguais nas duas.

CRITÉRIO DE PRONTO

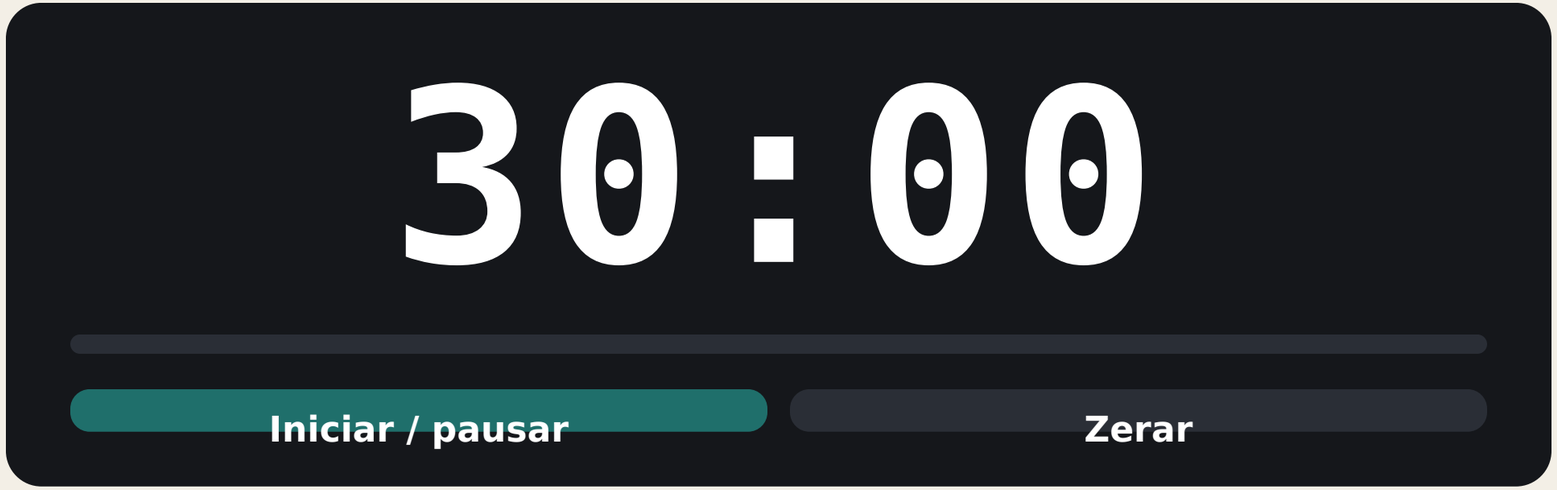
Para considerar entregue

HTML semântico, `alt` e `label` em tudo, uma coluna no celular. Validador sem erros, Lighthouse 90 ou mais e o `test_front.py` passando. Push feito.

Copiem o test_front.py da loja: ele funciona para qualquer pasta front/. O que não terminar hoje fica para casa; na Aula 12 entra o JavaScript.

LABORATÓRIO DA EQUIPE

Mãos à obra



- Criar `front/index.html` com a listagem do recurso.
- Criar `front/estilo.css` com o topo em Flexbox e os cartões em Grid.
- Criar `front/cadastro.html` com os campos do POST.
- Validador, Lighthouse e `pytest`.
- Commit e push.



O MAPA DE HOJE

Mapa completo

A loja ganhou vitrine e cadastro. Ainda estáticos: na Aula 12 eles reagem ao clique; na Aula 13, conversam com a API.

ONDE ESTAMOS AGORA

A primeira tela no ar

Telas no navegador	HTML e CSS hoje; JavaScript depois · Aulas 11 a 13	◂ HOJE
▲ fetch (Aula 13) ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✓ pronto
▲ chama ▼		
Serviço e repositório	as camadas entre a porta e o banco · Aula 10	✓ pronto
▲ usa ▼		
Domínio e banco	as classes e o loja.db · Aulas 5 a 10	✓ pronto

- As telas existem, mas os produtos estão **escritos à mão** no HTML.
- Na **Aula 12**: JavaScript, DOM e eventos. Os botões passam a responder.
- Na **Aula 13**: `fetch`. A vitrine busca os produtos na API.

FECHAMENTO

O que cada fase construiu

FASE	CONCEITO	O QUE PROVA
1-1	HTML, CSS e JS; elemento, atributo, documento	do arquivo à tela
1-2	header, nav, main, section, article, footer	a tag certa para cada caso
1-3	acessibilidade, WCAG, alt, label	o diagnóstico do trecho
2-1	seletor, classe, cascata, especificidade	o botão verde
2-2	modelo de caixa, box-sizing, variáveis	as quatro camadas
2-3	Flexbox, Grid, media query	Flexbox ou Grid
3-1	a vitrine semântica	`test_estrutura_semantica`
3-2	o visual em Flexbox e Grid	3 colunas no computador
3-3	formulário acessível e responsivo	66 passed e Lighthouse 90+

PARA ESTUDAR

Referências

BÁSICAS

DUCKETT, J. *HTML e CSS: projete e construa websites*. Rio de Janeiro: Alta Books, 2016.

MDN WEB DOCS. *Estruturando a web com HTML*. Mozilla. https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/Structuring_content

MDN WEB DOCS. *O modelo de caixa*. Mozilla. https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/Styling_basics/Box_model

MDN WEB DOCS. *Flexbox* e *Grids*. Mozilla. https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/CSS_layout

COMPLEMENTARES

W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. 2023. https://www.w3.org/TR/WCAG22/

BRASIL. *Lei nº 13.146, de 6 de julho de 2015* (Lei Brasileira de Inclusão), art. 63. https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13146.htm

W3C. *Markup Validation Service*. https://validator.w3.org/

GOOGLE. *Lighthouse: visão geral*. Chrome for Developers. https://developer.chrome.com/docs/lighthouse/overview

PARA DEIXAR ABERTO

Cola da aula

html

```
<header> <nav> <main> <section> <article> <footer>
<h1> um por página, títulos em ordem
<img src="..." alt="o que a imagem mostra">
<label for="x">Rótulo</label>
<input id="x" name="x" required>
<link rel="stylesheet" href="estilo.css">
```

css

```
* { box-sizing: border-box; }
.pai { display: flex; gap: 16px; align-items: center; }
.pai { display: flex; flex-direction: column; }
.grade { display: grid;
         grid-template-columns: repeat(3, 1fr); gap: 20px; }
@media (max-width: 700px) { .grade { grid-template-columns: 1fr; } }
```